

From Transportable Tablespaces to Pluggable Databases

Transporting data physically is faster and more predictable than a logical export/import. Therefore, it should be preferred for migration of large databases when possible. However, you may need more flexibility than that offered by Data Pump when moving a subset, or changing the version or platform.

Franck Pachot, Principal Consultant, dbi-services 

In Oracle 8i Transportable Tablespaces (TTS) were introduced, with the performance of physical transport for data, and the flexibility of logical export/import for metadata. If you look at some of the initial Application Programming Interfaces (APIs), you will see that this was initially called 'Pluggable Tablespaces'. The resemblance to Pluggable Databases is not a coincidence, as this 12c feature is an evolution of TTS.

If you have a very small database, using a logical export/import may be sufficient. But this method of importing is long and sometimes unpredictable. Even in direct-path, the import has to find free blocks, allocate and format them, and write rows. Then all indexes must be completely built. This takes time, with the need to read the table and sort rows, and requires lot of memory, and temp files. Finally, the index built may fail because of a lack of TEMP space. When you want the data movement to be faster and more predictable, you try to copy the data physically: copy the blocks without the need to modify them, and copy the index blocks in the same way as the table blocks, which are already built and usable.

History of Transportable Tablespaces

A database in Oracle 7 was monolithic. There were tablespaces to divide it, but their data was meaningless without the rest of the database for four reasons:

1. Datafile numbers present in all blocks (in ROWID) are database absolute file IDs.
2. Extent metadata is stored outside the tablespace, in the dictionary.
3. Undo information is stored in rollback segment, in the UNDO tablespace.
4. Metadata for your objects is stored in the dictionary.

In order to be able to transport the tablespaces, several improvements were implemented to make tablespaces more self-contained.

In Oracle 8.0, the ROWID identifies the datafiles with a relative number rather than an absolute number within the tablespace. This is very important because, otherwise, moving one tablespace would require modifications to all blocks.

In Oracle 8i, Locally Managed Tablespaces were introduced to manage the extents with a bitmap in the tablespace rather than the UET\$ table in the dictionary. This means the only physical information required to ship with tablespace transport is the segment header address.

This was enough to introduce TTS. But UNDO was still outside the tablespace. You need UNDO because, when you transport the tablespace, it holds some uncommitted data. From there comes the main limitation of TTS: the tablespace must be put in READ ONLY to be sure that there are no ongoing transactions.

Using Data Pump to transport tablespaces

The metadata is now exported from the dictionary with Data Pump (it was exp/imp at that time). Basically, Data Pump exports the Dynamic Data Link (DDL) to re-create the objects in the destination database to which the tablespace is moved. Below is an example where I export the tablespace USERS that I've put in READ ONLY before:

```
expdp system/oracle@//localhost/PDB1 directory=VAR_TMP
dumpfile=expdat.dmp transport_tablespaces=USERS exclude=table_
statistics,index_statistics;
```

In the log, you will see 'PLUGTS', which means 'Pluggable Tablespace':

```
Processing object type TRANSPORTABLE_EXPORT/PLUGTS_BLK
Processing object type TRANSPORTABLE_EXPORT/POST_INSTANCE/
PLUGTS_BLK
Processing object type TRANSPORTABLE_EXPORT/TABLE
Processing object type TRANSPORTABLE_EXPORT/CONSTRAINT/
CONSTRAINT
Processing object type TRANSPORTABLE_EXPORT/CONSTRAINT/REF_
CONSTRAINT
Master table "SYSTEM"."SYS_EXPORT_TRANSPORTABLE_01"
successfully loaded/unloaded
*****
*****
Dump file set for SYSTEM.SYS_EXPORT_TRANSPORTABLE_01 is:
/u01/oradata/tmp/expdat.dmp
*****
*****
Datafiles required for transportable tablespace USERS:
/u01/oradata/CDB1/PDB1/users01.dbf
```

At the end, the log mentions the tablespace's datafile(s) to move to the destination, and it has exported a dump file containing metadata only.

As with any Data Pump dump file, you can look at it by importing it with the 'sqlfile' option:

```
impdp system/oracle@//localhost/PDB1 directory=VAR_
TMP transport_datafiles=/u01/oradata/tmp/PDB1USERS2.dbf
sqlfile=sqlfile.sql ;
```

This does not import any objects, but generates the DDL. If you look at it, you will see all the DDL you may use to re-create the objects, with some additional attributes, for example:

```
CREATE TABLE "SCOTT"."EMP"
...
STORAGE(SEG_FILE 12 SEG_BLOCK 146 OBJNO_REUSE 73315
```

When you create a table, you just mention the tablespace and Oracle chooses where in the datafiles to create the segment header. Here we ship the datafile, and then the DDL must mention where the segment header is: its relative file number (SEG_FILE) and offset (SEG_BLOCK). The segments also have an identifier, the Data Object ID, which is found in each block; the same number will be re-used in the target database (OBJNO_REUSE). Do not try this syntax yourself: it can be used only internally by Oracle for bootstrapping and transporting datafiles.

Once you have shipped the datafiles to the destination (which takes the same time as a physical copy, but may also be as fast as a simple filesystem unmount/mount when you are on the same storage), you run the impdp command above, without the 'sqlfile'. You can change the tablespace name with REMAP_TABLESPACE and change the schema name with REMAP_SCHEMA. This means that you can clone a set of tablespaces within the same database.

This is a job for RMAN

Having the source tablespace in READ ONLY is a big limitation. You cannot use TTS to clone from production. When you want to clone the production, you use an RMAN DUPLICATE. This is also a physical transport, but less flexible than TTS: it's the whole database (even if you can skip some tablespaces); you overwrite the destination database; you must be in the same version; etc.

Then let's do the following:

- DUPLICATE from backup to a temporary database.
- Skip all tablespaces you don't want, except SYSTEM/SYSAUX (for metadata) and UNDO (for ongoing transactions).
- expdp TRANSPORT_TABLESPACE to get the metadata dump file.
- Remove all of this temporary database, except your datafiles.
- impdp TRANSPORT_DATAFILES to the destination.

So, you can transport tablespaces from production. It is a job for RMAN and since 10g RMAN has one command for that:

```
RMAN> transport tablespace PDB1:USERS
until time "timestamp '2017-12-04 12:00:00'"
auxiliary destination '/u01/oradata/tmp'
tablespace destination '/u01/oradata/tmp';
```

You will see RMAN doing everything: creating an auxiliary instance in the destination specified, restoring the tablespace you want to transport, restoring what is needed for metadata and recovery, setting it READ ONLY, exporting metadata. At the end, you get the datafiles, the dumpfile and a script to run the Data Pump import. You can look at the script, which is a nice way to see all the PL/SQL Application Programming Interfaces (APIs) behind impdp, or run the impdp yourself.

There's one thing to pay attention to in the RMAN log:

```
...
current log archived
RMAN-05026: warning: presuming following set of tablespaces
applies to specified point-in-time
List of tablespaces expected to have UNDO segments
Tablespace SYSTEM
Tablespace PDB1:SYSTEM
Tablespace UNDOTBS1
Tablespace PDB1:UNDOTBS1
Creating automatic instance, with SID='jCph'
...
```

The RMAN repository does not know where the UNDO was at the UNTIL TIME point-in-time and supposes that it is the same UNDO tablespace as the current one. If this is not the case, you can add the UNDO tablespace to the list of tablespaces to restore.

Self-contained tablespaces

As TTS can move a subset of the database, you need to be sure that you will not create inconsistencies when you plug them. They must be 'self-contained', meaning that there are no objects straddling two tablespaces that are not all included in the transport set. Such objects can be referential constraints, nested tables, domain indexes, materialised views, XML schema, and so on. Oracle provides a procedure to check this from the source, with error messages going to a collection that we can query through a view:

```
SQL> exec dbms_tts.transport_set_check(ts_list=>'USERS',incl_
constraints=>true);
PL/SQL procedure successfully completed.

SQL> select * from transport_set_violations;
no rows selected
```

It is recommended to check this before the export. You don't need to have the tablespaces in READ ONLY for that. Just be sure that no straddling objects are created between the check and the export.

Full Transportable Tablespaces

Now, only the metadata has to be transported logically. But when I'm talking about metadata for TTS, it covers only the metadata which is related to the data stored in the tablespace. This imports only the tables and the indexes. But TTS may be used to clone or migrate a database and there's a lot more metadata related to the schemas: user information, views, procedures, packages, etc. Until 12c you had to transport that separately as follows:

- Conventional Data Pump to create the users, roles, etc.
- TTS for tables and indexes
- Conventional Data Pump for all other objects

In 12c Full Transportable Tablespaces have been introduced to do both with the same command. You just add FULL=Y:

```
impdp system/oracle@//localhost/PDB1
full=Y transportable=ALWAYS
transport_datafiles=/u01/oradata/tmp/USERS.dbf
network_link=PDB1 directory=VAR_TMP logtime=ALL
```

This example uses a database link to get the metadata, so that you only have to transport the datafiles mentioned.

Cross-platform Transportable Tablespaces

One big advantage of TTS is flexibility because the data blocks are compatible between different versions, different platforms and even different endianness. When lowering downtime for a cross-endianness migration you cannot use a standby database, because the dictionary and the redo stream are endian-dependent. But you can transport tablespaces given that you use the RMAN CONVERT on the datafiles. This conversion does not modify all blocks (which would not then qualify for physical transport) but converts mainly the headers and UNDO.

You cannot apply REDO to them, but to lower the downtime, you can even apply incremental backups, so that you have only small files to copy and convert during the downtime. This is known as iXTTS (incremental Cross-Platform Transportable Tablespaces); there is more detail in MOS Note 2005729.1.

Pluggable Databases

Back to same endianness. We can transport user tablespaces physically, but the metadata must be exported logically. This can be very long when there are 50,000 tables as in some Enterprise Resource Planning (ERP) systems. The database is optimised for Data Manipulation Language (DML), not for DDL. This metadata resides in the SYSTEM and SYSAUX tablespaces. Then why not transport them? Because there's a lot of internal stuff there. You never export/import SYS objects because they are already there in the destination database. Doing so would mess up the whole dictionary, especially when going to a different version or character set.

This mix of system metadata with user metadata in the same dictionary tables has always been a problem and this is addressed in 12c multitenant architecture: system objects are in CDB\$ROOT, and user objects are in Pluggable Databases (PDBs). With multitenant architecture, thanks to the dictionary separation, we can now transport a PDB containing both metadata (in the PDB SYSTEM/SYSAUX) and data (in the PDB user tablespaces).

In 12cR1 you have the same limitation for READ ONLY because the UNDO is shared at CDB level and does not come with the transport of the PDB. Once the PDB is in READ ONLY, you can clone it within the same CDB or to another CDB if you mention a database links.

On the source database, I create a user with SYSOPER system role (there's a discrepancy between privileges mentioned in the documentation and what actually works):

```
SQL> grant connect, sysoper to C##DBA identified by oracle
container=all;
Grant succeeded.
```

On the destination, I create a database link to it:

```
SQL> create database link CDB1 connect to C##DBA identified by
oracle using '//localhost/CDB1';
```

And clone the database:

```
SQL> create pluggable database PDB2 from PDB1@CDB1 file_name_
convert=('CDB1/PDB1','CDB2/PDB2')
```

If you clone from 12cR1 to 12cR2 then the 12cR1 must have patch 18633374: COPYING ACROSS REMOTE SERVERS or you will get: "ORA-17630: Mismatch in the remote file protocol version client server". However, usually the PDB clone to a higher version CDB is faster than TTS. You still have to run datapatch (for higher patchsets) or catupgrd (for higher releases).

I haven't mentioned the ALTER PLUGGABLE DATABASE OPEN READ ONLY for the source because this is not required any longer in 12cR2 when in archivelog mode and in local undo mode.

Multitenant local undo

In 12cR2 the last limitation is removed: there is no need to put the source PDB in READ ONLY. The CDB can, and should, run in local undo mode. It is the default with DBCA, and you can check it with the presence of following property at TRUE:

```
SQL> select * from database_properties where property_name like
'%UNDO%';

PROPERTY_NAME PROPERTY_VALUE DESCRIPTION
-----
LOCAL_UNDO_ENABLED TRUE true if local undo is enable
```

With local undo, you can clone online: fuzzy datafiles are copied, redo is applied and ongoing transactions are rolled back thanks to the presence of the undo in the PDB. This means that you can clone a production PDB to get a test environment, with one command only.

Refreshable clones

When you clone production to test, you probably want to refresh it regularly. The clone I've created above uses the default 'refresh mode none', equivalent to:

```
create pluggable database PDB2 from PDB1@CDB1 file_name_
convert=('CDB1/PDB1','CDB2/PDB2') refresh mode none;
```

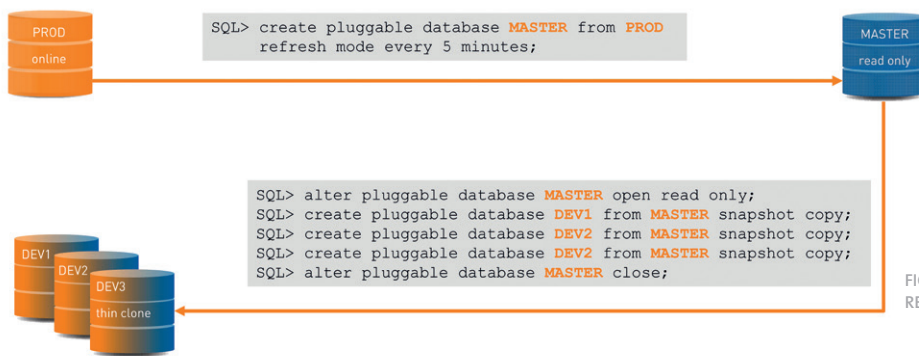


FIGURE 1: DBAAS THIN CLONES WITH A READ-ONLY MASTER

If you want to be able to refresh it later, applying the redo from the source to roll it forward, you can:

```

create pluggable database PDB2 from PDB1@CDB1 file_name_
convert=('CDB1/PDB1','CDB2/PDB2') refresh mode manual;

```

But this cannot be opened read–write. You can open it read only, but have to close it to refresh it:

```

23:17:52 SQL> alter pluggable database close;
Pluggable database CLOSE altered.
23:17:53 SQL> alter pluggable database refresh;
Pluggable database REFRESH altered.
23:17:55 SQL> alter pluggable database open read only;
Pluggable database OPEN altered.

```

You can also have the refresh scheduled, which will happen at the interval given that the clone PDB is closed:

```

create pluggable database PDB2 from PDB1@CDB1 file_name_
convert=('CDB1/PDB1','CDB2/PDB2') refresh every 5 minutes;

```

Then why have a read-only clone? You can use it as a master for further clones. And if you are on a file system that allows snapshots, such as ACFS, those can be thin clones. Figure 1 illustrates the idea: a master is refreshed frequently. When someone wants to provision one or several development environments, the master is closed, cloned as snapshot copy and re-opened.

However, in real life a read-only master is not sufficient. There are some passwords to modify, some database links to change, and probably some data to anonymise, scramble or mask. Then one intermediate copy is needed. This is illustrated in Figure 2.

Relocate PDB

You may realise that refreshable clones work with the same concept as a manual standby: copy the datafiles to initialise, and

then apply the redo to roll them forward continuously. It is like having an asynchronous standby at PDB level. And in the next release of the database, Oracle 18c will extend the refreshable clone syntax with the possibility of switchover and failover at PDB level. Because you can clone online, consider using that to move a PDB from one CDB to another, maybe on a different server and storage: stop the application, run the last refresh and re-open the application in the new location. This is implemented as ‘relocate’:

```

SQL> create pluggable database PDB2 from PDB2@CDB1
file_name_convert=('CDB1/PDB2','CDB2/PDB2')
relocate availability normal;
Pluggable database PDB2 created.

```

The ‘availability normal’ is the default. When you open the target PDB, the transactions on the source are killed and the application can then re-connect to the target PDB. If you are on the same server, with the same listener, this is transparent: the service is opened for the target CDB.

When changing, the connection string is needed to connect to the new CDB; you have the possibility of leaving a ‘tombstone PDB’ on the source CDB, which acts as a proxy and redirects connections to the new CDB. You achieve this with the following:

```

SQL> create pluggable database ... from ... relocate availability
max;

```

This is also called ‘near zero downtime’ because the sessions that are killed are able to re-connect immediately. Unfortunately, Application Continuity cannot be used here because the databases (CDB) are different.

Archive PDB

There is another use case for TTS: you can archive a set of tablespaces before dropping them. It is interesting that, when you need to read them later, you can plug them into a newer version.

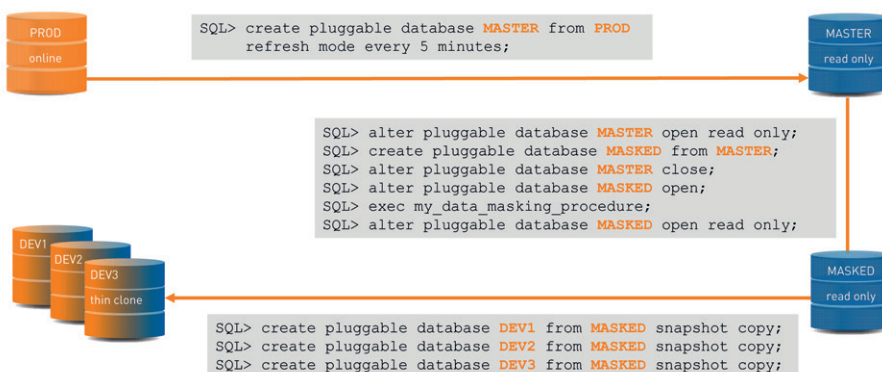


FIGURE 2: DBAAS THIN CLONES WITH DATA MASKING

This is more flexible than restoring a backup because then you must archive the Oracle software as well.

You have the PDB equivalent with PDB unplug. In 12cR1 the 'unplug' creates an xml file which you will ship with the datafiles:

```
SQL> alter pluggable database PDB1 unplug into '/var/tmp/PDB1.xml'
```

In 12cR2, simply change the '.xml' to '.pdb' and you will get a zip file containing both the xml file and the datafiles.

Use cases and conclusion

There are two use cases of interest. The first is for migrations. TTS is more flexible than PDB relocate because TTS allows endianness conversion and a different block size destination.

If you don't have these requirements, PDB movement is a good alternative, especially in Standard Edition where you cannot export TTS. A PDB clone can go to a different character set CDB when the target CDB is in AL32UTF8. When going to a newer version, you will choose between TTS or PDB depending on the number of objects you have: the former has to build the dictionary for your objects; the latter has to upgrade the dictionary for system objects.

When migrating from 11g to 12c multitenant you have two choices, again with PDB or TTS:

- Upgrade to 12c non-CDB, and then plug this non-CDB as a PDB. However, this requires running the noncdb_to_pdb.sql script on the dictionary, in addition to the catupgrd.

- Export from 11g with Full Transportable Tablespaces and import to a newly created PDB on the 12c CDB.

The second use case of interest is cloning and thin cloning, and refreshing all the test and development environments your agile developers require.

I use TTS a lot to replicate operational data to data warehouse Operational Data Store (ODS). This is a fast way of doing this. Combined with partition exchange, you can even do some incremental transport at partition level.

Then there are other reasons to export TTS or unplug a PDB when you need to archive or publish a read-only subset of data. Here again, TTS can be combined with exchange partition.

I have presented PDBs as an evolution of TTS, but one does not supersede the other. Find out about the different the tools and choose the one that best fits your needs. ■



ABOUT THE AUTHOR

Franck Pachot 🇫🇷

Principal Consultant, dbi-services

Franck Pachot (franck.pachot@dbi-services.com) is principal consultant, trainer and technology leader at dbi-services in Switzerland. With more than 20 years of experience in Oracle databases in all areas – from development, data modeling, performance, administration, and training – Franck is Oracle Certified Master 11g and 12c, an Oracle ACE Director and proud member of Oak Table. Franck is co-author of 'Oracle 12cR2 Multitenant' (<https://www.amazon.com/Oracle-Database-Release-Multitenant-Press/dp/1259836096>).

Blog: <http://blog.pachot.net>



@FranckPachot



<https://ch.linkedin.com/in/franckpachot>

orbit
reporting + analytics

ORACLE Platinum Partner

SELF-SERVICE REPORTING & ANALYTICS

For Oracle ERP & Cloud Applications

Bring Your Data to Life with:

- Real-Time Reports and Dashboards from All Key Data Sources
- Simplified Report Migration from Discoverer, Crystal Reports, and Other Reporting Tools
- 1,000+ Pre-Built Reports

Visit us at
UKOUG Stand
APPS 5



www.orbitanalytics.com